

Put your thesis on trial

Every other tool tells you why you're right. ThesisArena finds out if you are.

Whitepaper v1.0 · exported 21 September 2026 · figures read live at export

Contents

INTRODUCTION

Overview — What ThesisArena is, and the one idea it is built around.

The confirmation problem — Why on-chain tooling converges on agreeing with its user.

The stress-test principle — The single rule the architecture is organised around.

THE SYSTEM

How an investigation runs — The five stages, and what each one is allowed to do.

The four research modules — What each module reads, what it derives, and where it degrades.

Evidence Score and Data Coverage — Two separate questions that most dashboards quietly merge.

Stress conditions — How a sentence becomes six things that can be checked.

Monitoring and state — What happens after the investigation closes.

Cost-aware data strategy — The economics of the Nansen API, and how the pipeline is shaped around them.

NANSEN

Integration architecture — One client, one registry, one ledger.

Redistribution and compliance — Nansen's terms, enforced as code paths rather than policy text.

APPENDIX

Verification — How to check the claims in this document.

INTRODUCTION

Overview

What ThesisArena is, and the one idea it is built around.

Abstract

ThesisArena takes a stated crypto thesis, derives the conditions that would prove it wrong, and then tests those conditions against live on-chain data, continuously, until one of them breaks.

The design starts from an observation about how on-chain research actually fails. It rarely fails because the data was wrong. It fails because nobody wrote down, in advance, what would change their mind — so every subsequent piece of evidence gets read as confirmation. A tool that returns supporting evidence on request is not a research tool. It is a mirror.

This system inverts the default. Before it reports anything, it converts a thesis into **machine-checkable invalidation conditions**: a field, a comparator, a threshold, a duration. Then it commits to them. A condition can only be set *before* the evidence is weighed, never after. Monitoring then re-reads those conditions on a schedule, and the thesis has a state that can change without anyone asking it to.

The short version

- ◆ **Put your thesis on trial.**

Every other tool tells you why you're right. ThesisArena finds out if you are.

- 1 You state a thesis in plain language: “*\$TOKEN breaks out this week because smart money is accumulating.*”
- 2 The ticker is resolved against Nansen's live index to a concrete chain and contract.

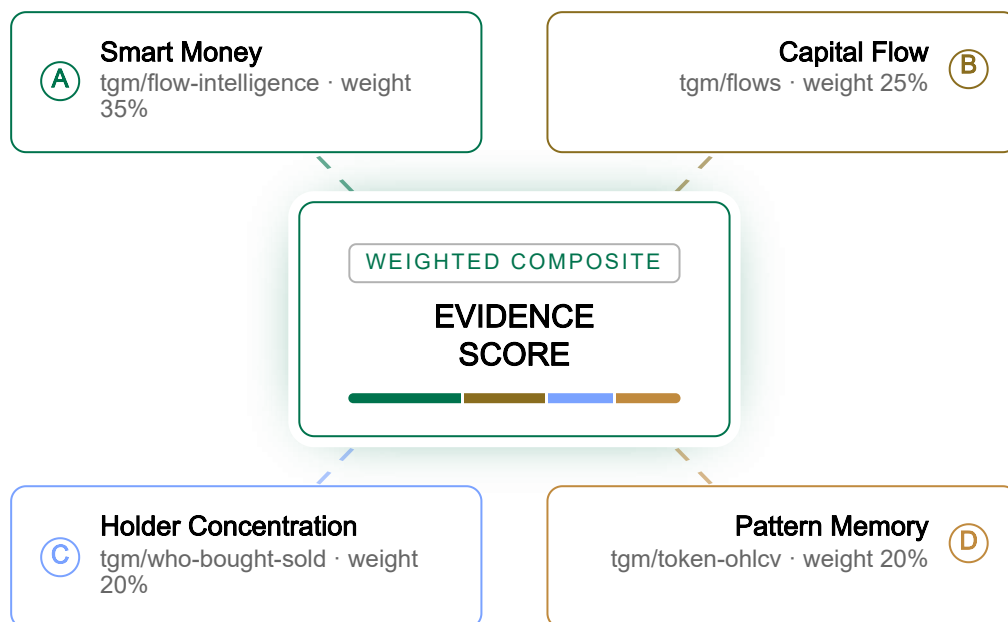
- 3 Four deterministic research modules interrogate it, and disagree out loud when they genuinely disagree.
- 4 Six stress conditions are derived from what was actually measured, each bound to a specific endpoint and field.
- 5 Those conditions are monitored. The verdict moves as real data arrives, and you find out when the thesis dies.

How it fits together

INVESTIGATION PIPELINE

Four modules run independently, combine into one composite, and commit to conditions before any verdict is shown.

ONE THESIS / FOUR MODULES
INDEPENDENT READS / ONE COMPOSITE



no model calls point-in-time provenance-bound

The evidence can be combined. The verdict is committed in advance.

What this document claims

This paper describes a system that exists and runs. Where it quotes a figure about the system's own behaviour (call counts, credit consumption, endpoint costs, module weights), that figure is read out of the running instance at the moment you load the page, and is marked **LIVE**. Where a number is a design constant chosen by judgement rather than derived from data, the paper says so explicitly at the point it is used.

What it is not

- **Not a price predictor.** The output is an Evidence Score, which measures the strength and internal consistency of the on-chain evidence. It is not a probability and does not translate into one.
- **Not a signal service.** There is no BUY, no SELL, no target. A thesis is the user's; the system only tries to break it.
- **Not a chatbot over an API.** No model call happens anywhere in the research path. The same inputs produce the same read, every time.
- **Not investment advice.** It is a research instrument.

INTRODUCTION

The confirmation problem

Why on-chain tooling converges on agreeing with its user.

Tools that agree with you

Ask any on-chain analytics product about a token you already like, and it will find you something encouraging. This is not dishonesty. It is the predictable result of an interface built around search.

A dashboard offers dozens of metrics across many timeframes. A user arrives with a prior. Somewhere in that space is a chart that agrees: a cohort that is accumulating, a window where flows turned positive, a holder count that ticked up. The user finds it, because finding it is what the interface is for, and leaves more confident than they arrived. Nothing in the product was wrong. The process was.

Why it happens structurally

- **The metric is chosen after the belief.** Any metric selected to answer “is my thesis working?” has already been filtered by whether it tends to say yes.
- **Timeframes are free parameters.** With six horizons available, a flow figure that is negative on one is usually positive on another. Choosing the horizon after seeing the data is choosing the answer.
- **Nothing is written down in advance.** Without a committed invalidation condition, no observation can ever count as disconfirming. It becomes noise, or early, or priced in.
- **There is no state.** A one-shot score is consumed and forgotten. A thesis that quietly stopped being true looks exactly like one that was never revisited.

What it costs

The cost is not a bad entry. It is the absence of an exit. A position held on a thesis that has already been invalidated by the data is the most expensive thing in the category, and it is

invisible precisely because no one defined what invalidation would look like. That is the gap this system is built to close.

INTRODUCTION

The stress-test principle

The single rule the architecture is organised around.

The rule

- ◆ A thesis that cannot be stated with a condition that would refute it is not a thesis. It is a preference.

Everything in the system follows from taking that seriously. The product does not attempt to determine whether a thesis is correct, which is unanswerable on a short horizon. It determines whether the thesis is *still standing*, which is answerable, repeatedly, from data.

Machine-checkable, not rhetorical

“Watch for weakening momentum” is not a stress condition; nothing can evaluate it. A condition in this system is a typed object bound to a specific endpoint and field:

```
{
  claim:    "Smart-trader accumulation is sustained",
  metric:   { endpoint: "tgm/flow-intelligence",
             field:    "smart_trader_net_flow_usd",
             params:   { timeframe: "1d" } },
  comparator: "<",
  threshold: 0,
  sustain:   "6h consecutive",
  severity:  "fatal",
  status:    "holding"
}
```

Because the condition names its own endpoint and field, re-checking it is a lookup rather than an interpretation. The checker contains **no model call**. That is what makes the verdict reproducible, and it is the difference between a research system and a conversation.

Why ordering matters

Conditions are derived from what the modules measured, and are committed before the composite score is computed. That ordering is the whole safeguard. A condition written after the verdict is known would be tuned, consciously or not, to a threshold the evidence already clears, which would make the entire exercise decorative.

▲ The constraint this puts on the product

It means the system will sometimes commit to a condition that is promptly met, and report that the user's thesis is invalidated minutes after they submitted it. That outcome is not a bug to be smoothed over. It is the product working.

The honest failure mode

A stress-test-first tool has a failure mode of its own: it can be too eager to declare a thesis broken, which is just confirmation bias with the sign flipped. Three things guard against it. Conditions carry a **sustain** window, so a single noisy reading does not trip one. Severity is graded, so only a **fatal** condition can move a thesis to Invalidated. And state is reversible — a condition that recovers moves back from **tripped** to **stressed**, which has been observed in live operation.

THE SYSTEM

How an investigation runs

The five stages, and what each one is allowed to do.

The five stages

STAGE	DOES	CONSTRAINT
Resolve	Extracts the ticker and resolves it to a chain and contract via live search	Spot contracts only
Investigate	Four modules gather evidence independently	No model calls
Debate	Raises a challenge where two modules genuinely disagree	Challenger attaches its own evidence
Commit	Derives six stress conditions from what was measured	Before the score is computed
Monitor	Re-reads those conditions on a schedule	Bypasses cache and fixtures

Resolution

The ticker is pulled out of the sentence and resolved against Nansen's live index, which costs nothing — `search/general` is a zero-credit endpoint, so asset search can be fully live without a budget consideration.

▲ Perpetual markets are filtered out

Nansen's index includes perp markets, and those rows carry the bare ticker in the address field. Hyperliquid's SOL market returns `address: "SOL"`. Passing that to a spot endpoint produces `Invalid address format`. Resolution therefore requires a real contract address, and the engine re-checks before any spot call. This was a live defect before it was a rule.

Disagreement

Most multi-agent systems stage a debate whether or not there is anything to debate, which produces the appearance of rigour and none of the substance. Here a challenge is raised **only** when two modules reach opposing stances, and the challenger's own evidence is attached to the message. The disagreement is inspectable rather than rhetorical, and when the modules agree, the interface says so and moves on.

Determinism

No language model runs in the research path. Modules are arithmetic over API responses; conditions are typed objects; the checker is a comparator. The same inputs produce the same read, which means a verdict can be re-derived and audited after the fact — and means the system cannot hallucinate a finding, because nothing in the path is capable of inventing one.

THE SYSTEM

The four research modules

What each module reads, what it derives, and where it degrades.

The module set

MODULE	READS	DERIVES
Smart Money	tgm/flow-intelligence	A flow term structure across six horizons and several wallet cohorts
Capital Flow	tgm/flows	Net token movement, buy pressure as a share of turnover
Holder Concentration	tgm/who-bought-sold	Net USD imbalance, top-3 share of volume, rotating-trader share
Pattern Memory	tgm/token-ohlcv	Analogues in the asset's own history, strictly point-in-time

Smart Money

Six calls at one credit each buy the signature signal: the same cohort netflows at 5m, 1h, 6h, 12h, 1d and 7d. The point is not any single figure but the **term structure**. If the short end is running slower than the long end, accumulation is decelerating even while it remains positive, and that is invisible in a single-timeframe read.

Magnitude is deliberately split in two. A bare logarithm of dollars saturates, since \$86K and \$86M both pin the scale, so half the weight is the flow's *share of all tracked cohort activity*, which does not saturate. This was a real defect. An \$86K flow once scored 100% confidence.

Capital Flow

Net token movement and buy pressure as a proportion of turnover. The endpoint refuses whole asset classes, native tokens and stablecoins among them, so the module catches the rejection and degrades to a cohort-level read rather than failing the investigation, and says on the page which class it hit.

Holder Concentration

Measured as **net USD imbalance across the whole trader set**, bounded to $-1 \dots +1$, alongside top-3 share of volume and the proportion of traders active in both directions.

■ A bug worth documenting

This module originally reported a count of net buyers as a ratio. It returned “100 of 100” on virtually every asset, because the endpoint returns the *top traders by volume* and those skew overwhelmingly to accumulators. The number was real, the inference was garbage, and it looked plausible enough to survive review. It was rebuilt on net USD imbalance, which cannot saturate that way.

Pattern Memory

Past windows in the asset’s own history with a similar return and volatility profile, and what happened next. Strictly point-in-time: only data available at the analogue date is used, so no forward information leaks into the comparison.

Graceful degradation

Confidence is capped by coverage. A module that received three candles cannot report 80% conviction, regardless of how clean those three candles look. Coverage is reported alongside the score rather than folded into it — see [Scoring](#).

THE SYSTEM

Evidence Score and Data Coverage

Two separate questions that most dashboards quietly merge.

Two numbers, not one

The **Evidence Score** says how strong and internally consistent the evidence is. **Data Coverage** says how much data actually backed the read. Merging them is the most common quiet dishonesty in the category.

A thin read that happens to be unanimous and a deep read that is unanimous are not the same finding, but a single blended number renders them identically. Reporting both means a reader can see that an 80 built on 30% coverage is a different object from an 80 built on 95%.

Module weights

COMPOSITE WEIGHTING

Read from the running system's WEIGHTS constant.



- Smart Money `tgm/flow-intelligence`
- Capital Flow `tgm/flows`
- Holder Concentration `tgm/who-bought-sold`
- Pattern Memory `tgm/token-ohlcv`

▲ These weights are judgement

They are not calibrated. Nothing has been backtested to justify 35/25/20/20 over any other reasonable split. They reflect a view that cohort flow is the most informative single signal available at one credit — a view, not a result.

Independent corroboration

Every composite is combined with a substantial independent source, [DeFiLlama ↗](#), read separately for chain TVL and stablecoin supply. It is displayed in its own panel, visually distinct, and explicitly labelled *not Nansen data*. Corroboration nudges the composite; it does not dominate it.

This is simultaneously an analytical choice and a licensing requirement. See [Compliance](#).

Not a probability

- An Evidence Score of 72 does not mean a 72% chance of anything. It is an index of evidential strength and agreement, on a scale defined by this system and comparable only within it. It is never rendered as a probability, a confidence interval, or a recommendation, and the scoring code has no path that would produce one.

THE SYSTEM

Stress conditions

How a sentence becomes six things that can be checked.

Anatomy of a condition

Six conditions are committed per investigation. Each names the endpoint and field it reads, a comparator, a threshold, a sustain window and a severity — everything needed to evaluate it later without interpretation.

FIELD	PURPOSE
metric	Endpoint, field and params — makes the condition re-readable
comparator	Less than, greater than, or crosses
threshold	The value that constitutes refutation
sustain	How long it must hold, so noise does not trip it
severity	fatal, major or minor
status	holding, stressed or tripped

How they are derived

Conditions come from what the modules actually measured on this asset, not from a template. If Smart Money measured a positive 7-day smart-trader netflow, the condition binds to that field and sets the threshold relative to the observed value. An asset where a module degraded produces fewer conditions bound to that module, rather than a placeholder condition that would always hold.

Severity

- **Fatal.** If met, the thesis is invalidated. These bind to the mechanism the user actually claimed.
- **Major.** Meaningful pressure. Enough to move the thesis to Under pressure, not enough to end it.
- **Minor.** A supporting observation that has stopped supporting.

Anchoring

▲ Tautological thresholds

An early version anchored a price condition to the current return, which made it trivially true at the moment of writing and therefore worthless. Price conditions now anchor to a 21-day swing low : a level that existed before the thesis did, and that the thesis can genuinely fail against.

THE SYSTEM

Monitoring and state

What happens after the investigation closes.

The monitoring loop

A monitored thesis re-reads **only the metrics its conditions are bound to**. Nothing new is invented to monitor, and no module is re-run wholesale. A core check costs 2 credits; every sixth cycle runs a deeper check at 5.

♦ Monitoring never reads from cache

The monitor sets `cacheTtlMs: 0` and bypasses the fixture layer entirely. A cached answer would defeat the entire purpose — the question being asked is specifically “has this changed since last time?”

If the key runs out of credits, the job auto-pauses and records why, rather than silently reporting that all conditions still hold. A monitor that cannot read is distinguishable from a monitor that read and found nothing.

States

THESIS STATES

The state is derived, not stored — recomputed from the conditions and module stances on every read.

● Supported	SUPPORTED	Evidence supports the thesis and no decisive contradiction is present
● Mixed	MIXED	Evidence is split across the research modules
● Under pressure	UNDER_STRESS	One or more conditions show stress, but the thesis has not failed
● Challenged	CHALLENGED	The evidence is materially contradicting the thesis
● Invalidated	INVALIDATED	A fatal condition has been met

Precedence

Order matters, and getting it wrong produces a genuinely misleading interface. A fatal trip outranks everything. But zero module support outranks a merely stressed condition — if no module supports the claim, the thesis is Challenged, not Under pressure.

```
if (fatalTripped) return 'INVALIDATED';

// No module supports the claim: the modules themselves contradict it,
// and that outranks "a condition is under pressure".
if (support === 0 && challenge > 0) return 'CHALLENGED';

if (anyTripped || anyStressed) return 'UNDER_STRESS';
```

Before this ordering existed, a thesis with zero support and four challenges rendered as Under pressure, which reads as “intact but losing force” when the correct reading was “the evidence contradicts this.”

Observed in operation

The central promise is that a verdict changes on its own. It does. Two monitored theses moved from Under pressure to Invalidated on live data after their investigations had closed, and a third recovered from **tripped** back to **stressed**, confirming the state machine is not a one-way ratchet toward bad news.

THE SYSTEM

Cost-aware data strategy

The economics of the Nansen API, and how the pipeline is shaped around them.

The premise

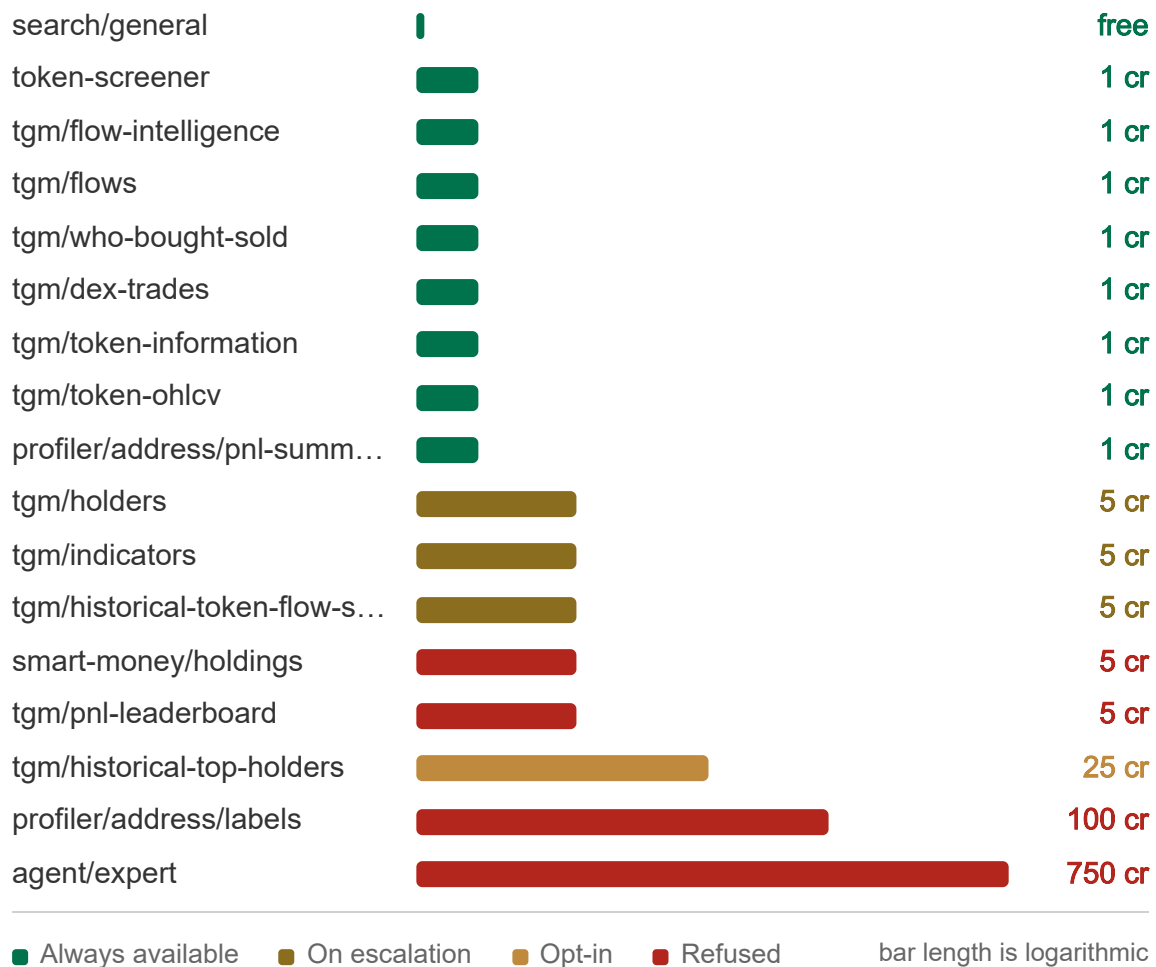
ThesisArena is designed around the economics of the Nansen API. Credit costs vary significantly across endpoints, so the investigation pipeline is deliberately structured to spend credits where they provide the most research value.

This is part of the product architecture rather than an implementation detail. The goal is to maximise evidential value per API credit, and that objective shapes which endpoints exist in the research path at all.

Cost across the surface

CREDIT COST BY ENDPOINT

Read from the endpoint registry the client enforces at runtime.



The spread exceeds 500×. A naive implementation reaches for the richest-sounding endpoint and discovers it has spent 750 credits on a single call. The registry makes cost a property of the endpoint that the client consults before every request, rather than something a developer is expected to remember.

Tiered request strategy

- **Low-cost endpoints** run as the default investigation layer.
- **Mid-cost endpoints** are invoked when the initial evidence is inconclusive.
- **High-cost endpoints** require explicit conditions before they are used.
- **Blocked or non-redistributable endpoints** are excluded from the public research path entirely.

TIER	COST	WHEN IT RUNS
Low	0–1 credits	Always
Mid	5 credits	Only when the cheap tier is inconclusive
High	25 credits	Explicit opt-in per call
Blocked	never spent	Throws; never reached

The blocked tier matters more than it looks. Endpoints that are ruinously expensive or prohibited for display do not merely go uncalled by convention. The client raises rather than calling them, so avoidance is structural rather than a matter of discipline.

The request ledger

Every live Nansen request is recorded with its endpoint, request id, credit usage, status, latency and remaining balance. Rejected requests are billed as zero credits, and cache or fixture replays do not count as live network calls.

♦ Why the distinction is enforced in the schema

A **CHECK** constraint restricts the source column to **live**, **cache** or **fixture**. Counting a replay as an API call would be the easiest possible way to inflate the numbers, so the database refuses it rather than leaving it to convention.

Live consumption

Measured from the ledger on this instance, right now: **LIVE**

LIVE CALLS 4,648 real network requests only	CREDITS CONSUMED 4,522 0.97 per call
CREDITS REMAINING 15,445	ZERO-COST REPLAYS 21

An average below the headline rate is the tiering working. The workhorse endpoints are the cheap ones, and the expensive tier is reached only on purpose.

Integration architecture

One client, one registry, one ledger.

A single chokepoint

Every request goes through one client. Cost lookup, budget enforcement, rate limiting and ledger recording all live there, so none of them can be bypassed by adding a call somewhere new. There is no second path to the API, deliberately, including no MCP path, because a parallel route would break the counting guarantee the whole submission rests on.

The ledger

Every request appends a row: endpoint, credits, Nansen request id, remaining balance, status, latency, and, critically, its `source`.

```
source TEXT NOT NULL CHECK (source IN ('live','cache','fixture'))
```

That constraint is enforced by the database rather than left to convention. Only `live` rows are real network calls against the API key, and only those are ever counted. See [Verification](#).

Record and replay

A response is recorded once and replayed at zero credits thereafter. This is how the application was developed against a trial key without burning the budget on iteration.

◆ Fixture identity ignores dates

The cache key canonicalises ISO date strings out of the request body. Without that, a rolling 7-day window makes every request unique and the entire fixture set expires nightly. Which it did, until the hash was changed.

Credit accounting

The client trusts `x-nansen-credits-used` from the response header rather than assuming the endpoint's list price. A rejected request bills zero, and an early version that charged itself for its own 422s reported a credit burn that never happened.

```
const usedHeader = res.headers.get('x-nansen-credits-used');
const parsed = usedHeader === null ? NaN : Number(usedHeader);
const used = Number.isFinite(parsed)
  ? parsed
  : res.ok ? spec.credits : 0;
```

Redistribution and compliance

Nansen's terms, enforced as code paths rather than policy text.

What the terms require

Nansen permits derived analysis but prohibits republishing its proprietary signals. Meeting that is not a matter of avoiding a few endpoints; it constrains what may be rendered, and it requires the output to be meaningfully combined with a substantial independent source.

▲ This reshaped the product

An earlier design let a user click each researcher and inspect the raw Nansen evidence behind it. That is precisely what the terms prohibit. The resolution, restricted data entering only as a weighted term inside a composite, turned out to be the better product, and it is also what Nansen's own documentation gives as a compliant example.

Redistribution classes

CLASS	MEANING	MAY BE RENDERED RAW
allowed	Freely redistributable	Yes
attribution	Redistributable with attribution	Yes, attributed
restricted	Composite input only	No
prohibited	Never called, never rendered	No

How it is enforced

- **4 endpoints are blocked** in the registry. The client raises rather than calling them.

- **2 endpoints are restricted.** Values contribute weighted terms to a composite and are tagged *derived* wherever they surface.
- **A render guard, `assertDisplayable()`,** throws if raw values from a restricted or prohibited class reach the render layer. It is covered by a test, which is the compliance proof.
- **An independent source is required,** not optional. Every composite is combined with DeFiLlama, shown in its own panel and labelled *not Nansen data*.

Attribution

Powered by Nansen API appears in the footer and on every shareable result card, so attribution travels with the output rather than living only on the site.

APPENDIX

Verification

How to check the claims in this document.

What counts as a call

Only a real network request against the Nansen key counts. Cache hits and fixture replays are recorded, displayed, and excluded.

This is the kind of claim that is easy to make and rarely enforced, so it is enforced in the schema rather than in a function someone could forget to call. Call volume comes from investigations users run and from monitoring checks re-reading conditions committed to in advance across a portfolio of distinct theses — not from a loop around a cheap endpoint.

Current ledger

Read from this instance at page load: • LIVE

LIVE CALLS 4,648 counted	SUCCESSFUL 4,639 HTTP 200
FAILED 9 billed 0 credits	EXCLUDED 21 cache hits and replays

The full request ledger, with per-endpoint and per-source breakdowns, is at [Analytics](#).

Exporting the evidence

```
npx tsx scripts/export-ledger.ts
```

```
export/nansen-requests.csv  every live call with its Nansen request id  
export/usage-summary.json  totals, per-endpoint and per-context breakdown
```

Each row carries the request id Nansen returned, so the ledger is checkable against Nansen's own records rather than being taken on trust.

Tests

The suite asserts the claims this product makes about itself. Several tests encode defects that reached the interface and were fixed: the score clamp that made every broken thesis score exactly 35, the status precedence that mislabelled contradicted theses, the perp-address filter, and the negative-zero formatter. A regression there is a regression in the product's honesty, not just its output.